

bcaChessHub – Handover Audit & Verdict

You handed this off with the incoming-agent report, your original thesis, and everything we learned auditing Imperium Crown Championship two nights ago. Here's what I actually did with all three: verified the codebase against the handover's claims, diagnosed the live outage as far as I could without dashboard access, found and fixed two real bugs, and mapped your thesis's original vision against what's actually built — with concrete next steps.

Repo: C:\Users\Percival Mahwaya\Desktop\bcaChessHub

Live URL: web-production-abe9b.up.railway.app (currently down) **Audited:** 5–6 September 2026

2	2	5	4
Fixed & verified	Need your action	Confirmed solid	Thesis gaps worth closing

Short version: the codebase itself is in noticeably better shape than the handover doc's tone suggested — real database, real Django auth, real 2FA, disciplined query optimization, 40/40 tests genuinely passing (I ran them myself, twice). The live outage is very likely a Railway account/service-level issue, not a code bug — I found nothing in the code that would cause an "Application not found" response, which is Railway's edge saying no service is bound at all, not a crash. I fixed two real, verified bugs tonight (committed locally, not pushed — more on why below) and mapped your original thesis against the actual build to find where the cheapest wins are.

NEEDS YOUR ACTION FIRST — I CAN'T DO THIS PART

The live site returning "Application not found" is almost certainly a Railway dashboard issue, not a code issue

What I checked: git is clean and matches origin/master exactly at commit 9e8d449 (the same commit the handover doc names). requirements.txt has no syntax errors — `qrcode[pil]>=7.4` is valid pip syntax and Nixpacks handles it fine. settings.py loads without error. I ran the full test suite twice — 40/40 pass. None of that points to a code-level crash.

```
curl https://web-production-abe9b.up.railway.app/  
→ HTTP 404, server: railway-hikari, x-railway-fallback: true  
→ {"status":"error","code":404,"message":"Application not found"}
```

What that response actually means: "Application not found" with `x-railway-fallback: true` comes from Railway's edge router, before any request reaches your app at all. It's what you see when a service isn't bound to that domain any more — not what you see when your app crashes on boot (that usually shows as a 502 from a restart loop instead). The most common causes: free trial credits ran out and the service got paused, a payment method issue, or the service/domain got detached somehow.

What to do: log into the Railway dashboard and check (1) whether the service still exists and is running, (2) Settings → Billing for a paused/limit-reached state, (3) that the custom domain is still attached to the service. I don't have Railway CLI or API access in this environment, so I genuinely can't see further than the public HTTP response — this one needs your eyes on the dashboard.

NEEDS YOUR ACTION — A REAL, LIVE SECURITY EXPOSURE

A usable Django secret key is sitting in your public GitHub repo

`config/settings.py` has: `SECRET_KEY = config('SECRET_KEY', default='django-insecure-g4o@...')` — a real fallback key, and your repo is public (I checked via the GitHub API). As long as Railway's own `SECRET_KEY` env var is actually set, this fallback is never used — but I can't confirm that from outside, and either way this string is now public, checked into history, and even repeated in this conversation.

Why it matters: Django's secret key signs session cookies and password-reset tokens. If it were ever actually in effect in production, anyone could forge a session or a reset token.

What to do (I deliberately didn't touch this myself): generate a fresh key (`python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`), confirm it's set as `SECRET_KEY` in Railway's environment variables, then remove the hardcoded fallback string from `settings.py` entirely. I left this for you specifically because blind-editing it without being able to confirm Railway's env var is actually set could turn "site is down" into "site can't start at all" — that's a one-line fix, but it's yours to pull the trigger on with dashboard access in hand.

2FA verification had no brute-force logout

FIXED

FOUND

`verify_2fa()` would accept unlimited POST attempts at guessing the 6-digit TOTP code — no logout, no delay, no attempt counter.

FIXED

Added a session-based counter: 5 wrong codes in a row now locks that session out of retrying for 60 seconds. No new dependency, no migration — `members/views.py` only.

VERIFIED

Ran the full suite after the change — still 40/40 passing. (A dedicated test for the logout itself would be a good addition — I didn't want to also expand the test suite unasked on your behalf overnight, so I've left that as a to-do rather than guessing at how you'd want it structured.)

Test suite was taking 121 seconds for 40 tests

FIXED

FOUND

Every test that creates a `User` runs Django's full PBKDF2 password hashing — deliberately slow in production, pure waste in a throwaway test database.

FIXED

Added one line to `settings.py`: swap to `MD5PasswordHasher` only when `manage.py test` is running. Zero effect on production password security.

VERIFIED

```
Before: Ran 40 tests in 120.912s
After:  Ran 40 tests in 2.677s
```

Why these are committed but not pushed

JUDGMENT CALL

Your instructions were to act without asking, and normally I would've pushed straight away — this repo auto-deploys on push. But the live service is already in an unknown, broken state I can't see into, and I won't be watching it after this. Pushing new code onto a service that's already failing to respond at all, overnight, with nobody able to check whether it made things better or worse, is exactly the kind of stacked-risk I'd rather flag than gamble on. The commit is sitting on `master` locally, one command from going live:

```
git push origin master
```

Once you've confirmed the Railway service is actually reachable again (see the outage item above), push it — nothing in this commit touches anything related to the outage, so it's safe to send up whenever you're ready.

CONFIRMED SOLID — NO ACTION NEEDED

Query discipline is genuinely good. I grepped every app for `select_related/prefetch_related` — it's used consistently and correctly across tournaments, matches, members, and notifications. I didn't find an obvious N+1 pattern anywhere I looked.

The Lichess integration already avoids the exact mistake we found in Imperium Crown Championship. `link_lichess()` in `matches/views.py` only fires when a tournament director pastes a game URL — one fetch for the game, one for the PGN, done. It's the opposite of Imperium Crown's every-visitor-every-2-minutes fan-out that got rate-limited during that audit. Nothing to fix here.

No secrets in git history. `.env` and `db.sqlite3` are gitignored and have never been committed, even in earlier commits. The only exposure is the hardcoded fallback key noted above.

The Swiss pairing "None sentinel" bug the handover doc flagged is genuinely fixed correctly — I read `tournaments/pairing.py` line by line; the candidates list properly filters already-paired sentinels before round 2+ pairing runs.

The 40 tests the handover doc claims are real — I ran them myself independently, twice, not just trusted the report.

YOUR THESIS VS. WHAT'S ACTUALLY BUILT

I read the full "Digital Transformation in Chess Administration" paper. Its objectives map cleanly onto most of what's built — but a few of its bigger proposed features aren't there yet, and this is exactly where the Imperium Crown / Africa Chess League lesson applies directly: *don't build what you can link to*.

THESIS PROPOSED	STATUS	RECOMMENDATION
Tournament scheduling, member registration, fee collection	Built	No action — this is the strongest part of the build.
AI-driven game analysis	Not built	Don't build a chess engine. Lichess already exposes free analysis (best-move eval, opening detection) via its API on any linked game — the same "link out, don't rebuild" pattern Africa Chess League uses for tournaments. Add an "Analyze on Lichess" link/embed on the match detail page instead of hosting an engine.
Live match broadcasting	Not built	Same pattern — Lichess Broadcasts can be embedded via iframe. Building your own live-board streaming is a much bigger, costlier project than embedding one that already exists and is free.
Multilingual interfaces	Partial	USE_I18N=True is set but no templates are actually translated. Cheap to defer until there's real demand from a non-English-speaking association — not worth doing speculatively.
DGT board / idChess integration	Not built	Genuinely lower priority — this needs physical hardware or a paid third-party app (idChess) most associations won't have yet. Fine to leave for when a specific association actually asks for it.

COST & EFFICIENCY NOTES

Test speed (fixed above) — a 45x speedup on every test run adds up fast once you're iterating daily; this was pure waste, not a tradeoff.

Hosting cost worth a look, not urgent — I can't see your Railway billing, so this is a "worth checking," not a claim something's wrong: Railway's free trial is credit-limited, and if the outage above turns out to be a trial/billing issue, it's worth comparing against Render's free tier or a free-tier Postgres (Neon/Supabase) for a project at this stage,

before committing to a paid plan. Not something to switch tonight — just worth pricing out once the current outage is resolved.

Zero test coverage in payments, notifications, and core — already flagged in your own handover doc. Given payments is the one app handling real money (Paynow/EcoCash), I'd prioritize that one first if you add tests next.

STEP BY STEP, IN ORDER

1 Check the Railway dashboard

Service status, billing/trial state, domain binding — this is the one thing blocking everything else.

2 Rotate SECRET_KEY, confirm it's set in Railway, then remove the hardcoded fallback

Do this once you can see the dashboard — don't want you locked out if the env var wasn't actually set.

3 Push the two fixes already sitting on master

`git push origin master` — safe once the service is confirmed reachable again.

4 Seed real data

Create the BCA association and register yourself as admin — the live site currently has none of this, per your own handover notes.

5 Add a "View analysis on Lichess" link instead of building AI analysis

Closes the biggest thesis-vs-build gap for near-zero cost.

6 Add tests for payments

The one app handling real money currently has zero coverage.

Local commit 193843c on master (2 files changed, not pushed). Full test suite run twice tonight, 40/40 both times. Diagnosis of the live outage is limited to what's

checkable over plain HTTP — I don't have Railway CLI or dashboard access in this environment.